OpsHub — Solution

Category: WEB | Points: 500 | Author: Jerit3787

Overview

A three-service pivot chain. The public edge portal has an SSRF in its health-check tool whose denylist forgets `172.16.0.0/12`. That reaches two internal, unshipped services: a cloud-metadata clone that hands out a read-scoped token + the deploy API address, and a deploy API whose create endpoint is vulnerable to mass assignment, letting a read-scoped user smuggle `config_source=production` onto a self-owned draft and render production secrets (the flag).

Steps

1. Log into OpsHub (`opshub` / `opshub2026`). Find the Health Check URL fetcher.
2. Probe the health-check (no source is shipped): loopback, `10/8`, `192.168/16`, `169.254/16` (and the public internet) are refused with "url not allowed", but **`172.16.0.0/12` addresses are fetched**. The dashboard names the internal services `metadata` and `deploy`.
3. Through the SSRF, scan `172.20.0.0/24` (low addresses, ports 80/8000). Find metadata at `172.20.0.3:80` and deploy at `172.20.0.4:8000`.
4. Walk the metadata IMDS tree: `/computeMetadata/v1/instance/service-accounts/default/token` -> a `deploy.read` token; `/computeMetadata/v1/instance/attributes/deploy-endpoint` -> the deploy URL.
5. Probe the deploy API. `GET /api/deployments/dep-900/config` -> 403 (IDOR dead end); the token is opaque (no forgery). But prod deployments show `config_source: production`, and `create` echoes/accepts that field.
6. Mass-assign it on your own draft: `GET /api/deployments/create?token=<t>&name=x&config_source=production`.
7. Render your own draft: `GET /api/deployments/<id>/config?token=<t>` -> the flag.

See `solve.py` for the fully automated chain (edge SSRF only).

Red Herrings

- **IDOR** on other deployments' `/config` — correctly scope-gated (403).
- **Token forgery** — the token is opaque and validated server-side; not a JWT.
- **`scope=` query tampering** on the deploy API — ignored; scope is server-side.
- **Metadata `scopes` leaf** advertises `deploy.write`/`deploy.admin` tiers you cannot obtain directly — the point is you never need them.

Flag

```
gctf26{my_0th3r_c4r_1s_y0ur_m3t4d4t4_s3rv3r}
```